



Business Central Agentic Development Framework

Simple Enough for Functional Consultants.
Robust Enough for Pro Developers.

Created by AJ Ansari, Microsoft MVP

Built so a Functional Consultant can build AL the right way

This Agentic Development Framework was created to help Business Central Functional Consultants use AI to build AL extensions and apps the **right** way — though professional AL developers will find it just as useful.

Two editions: the **full framework** for substantial projects, and **Lite** for smaller ones that need to move fast. Both apply the same AL standards.

Either way, the routine grounds the agent in explicit rules, official references and quality tooling rather than AI guesswork alone:

OnlyCopilotFans AL Development Standards Guide The AL rules the routine applies at each step, fetched in automatically.
AL MCP Server Build, publish, symbol and diagnostic tools, driven directly.
BC Base App on Microsoft Learn Official references instead of half-remembered platform behavior.
BCQuality A curated quality knowledge base, snapshotted per project.
OnlyCopilotFans BC AL Patterns A portable pattern library

Four phases. Fourteen steps. A gate on every one.

DEFINE

Scope the problem, fill in parameters. PRE-01 · PRE-02 · 01



DESIGN

FRD, plus a self-sufficient TDD. 02 · 03 · 04



BUILD

Generate AL, lint, then compile and package. 05 · 06 · 07



PROVE

Gap-fit, review, document, release. 08 · 09 · 10 · 11 · 12



In Production, Documented

ALL ALONG — continuous discipline runs underneath every one of those four phases.

Every shape means one thing



Start or end point of a flow



An action, step, or process



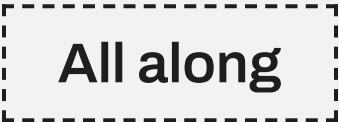
Input or output — data, a document



An exit gate — met before the next step starts



A decision point



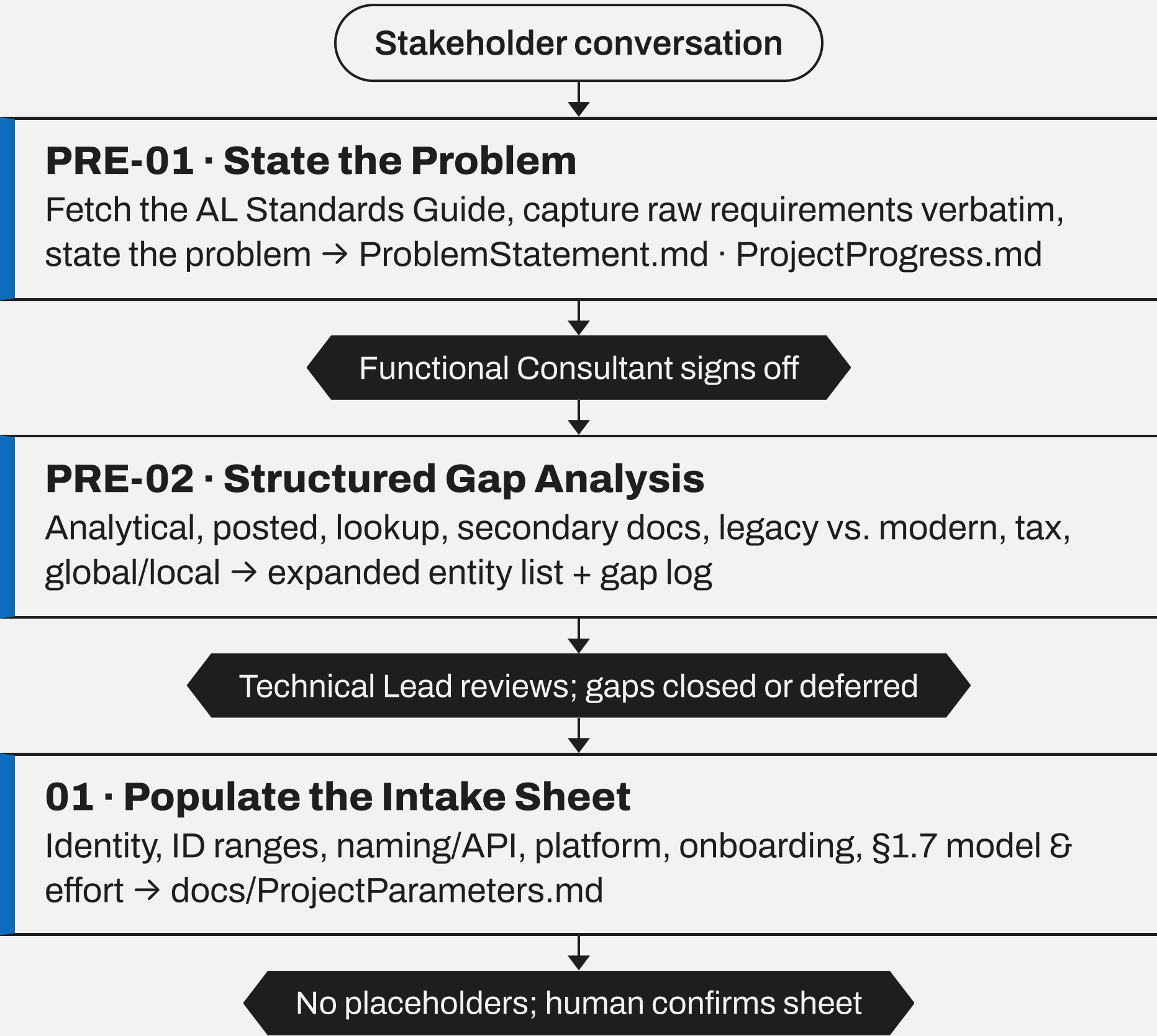
A cross-cutting concern, running underneath

Main
Writes the code

Light
Pre-flight only

Reasoning
Drafts, reviews

DEFINE — ask, don't infer



DESIGN — what, then how

02 · Craft the FRD REASONING

Business language — what and why, not how. Validate against DEFINE
→ FRD.md

Dev Manager sign-off; every entity accounted for

03 · Craft the TDD REASONING

System identity, module grouping, batch plan, per-object spec,
permission sets — must be self-sufficient → TDD.md

Technical Lead sign-off; self-sufficiency check

04 · Sanity Check REASONING

Can BC actually do this? Does the TDD implement the FRD? →
SanityCheck.md

0 blocking issues

The Main role integrates every draft and carries it to the human. The Reasoning role never owns sign-off.

BUILD — compile, package, iterate

05 · Plan the Code

Batch order, scaffold, AL MCP Server + BCQuality + Patterns Library, pre-flight checks written

Batch order agreed; scaffold complete, not compiled

06 · Code Generation repeats per batch

Main approves and generates, then runs pre-generation pre-flight.
Light runs post-generation pre-flight and permission-set coverage, and reports back.

↻ loop until every planned batch exists — no compile here

Every batch pre-flight clean; coverage verified

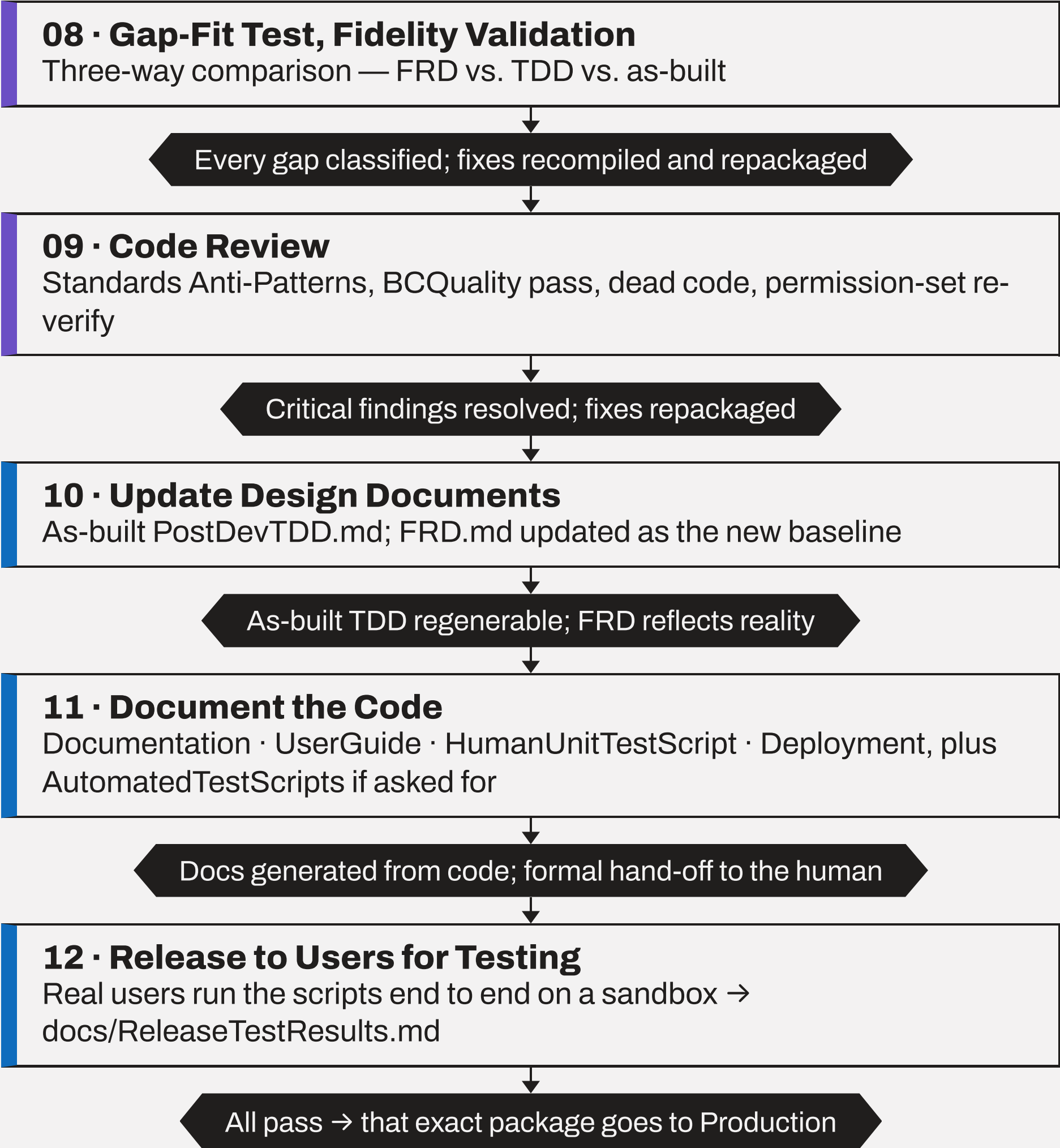
07 · Compile and Package, Troubleshoot, Iterate

One mandatory compile-and-package opens the step. **Reasoning** diagnoses root cause — patterns library first, then the three questions — **Main** fixes.

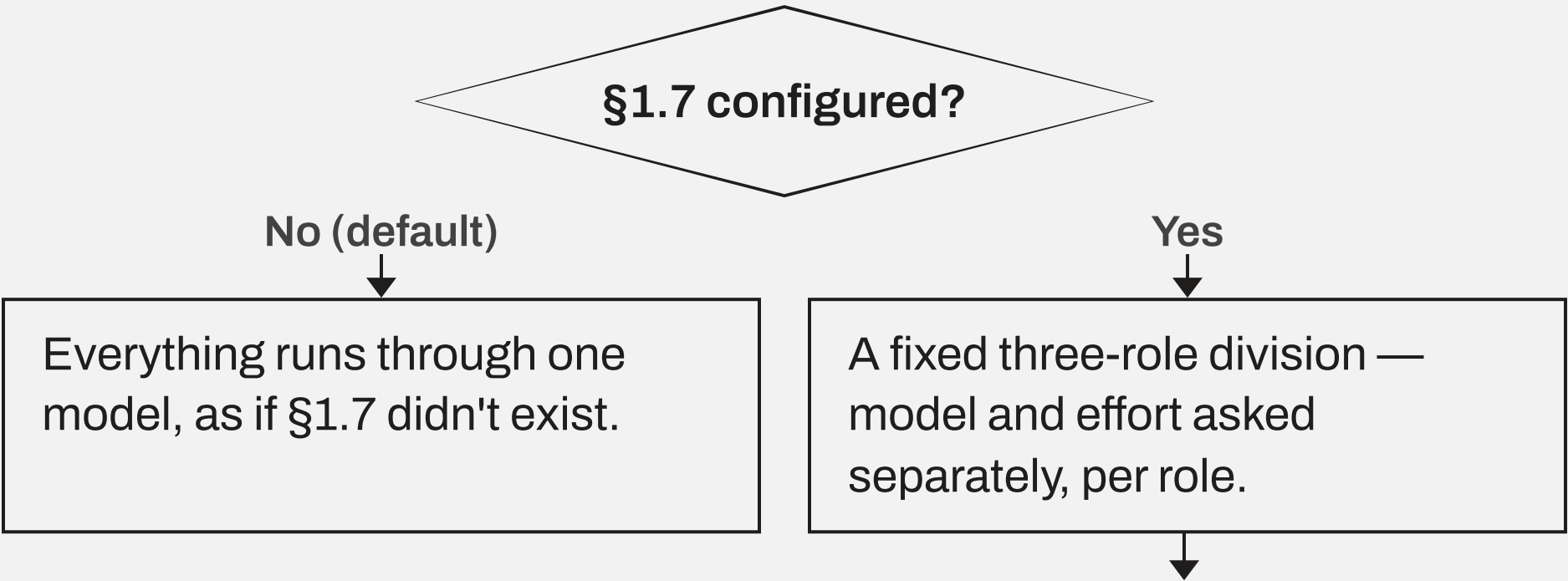
↻ publish → test → diagnose → fix → repack → redeploy — the rhythm of the rest of BUILD and PROVE

0 errors / 0 warnings

PROVE — five steps to production



Model & Effort Assignment (§1.7)



MAIN ROLE
Primary Agent · a model like Claude Sonnet · effort High (default)
All code generation and every code edit, including applying what Light and Reasoning report. Owns ChangeLog, Object Register, ProjectMemory, TestingFeedback.md.

LIGHT ROLE
Sub-Agent · a model like Claude Haiku · effort High (default)
The post-generation pre-flight pass only — props, Rec.-qualification, dead code, indentation, coverage, symbol verification. Reports; never edits.

REASONING ROLE
Sub-Agent · a model like Claude Opus · effort High (default)
Sanity Check, Gap-Fit Test, Code Review, FRD and TDD authorship, root-cause troubleshooting. Drafts and diagnoses; never edits code or continuity docs.

ALL ALONG — every phase, every step

Document Nineteen required documents, kept current — never retroactively.	Track Changes Every FRD/TDD deviation logged before the next batch.
Retain Explanations Who decided, and why — not just the outcome.	Testing Feedback Log Verbatim findings from Step 07 onward, triaged explicitly.
Project Memory docs/ProjectMemory.md — an anchor, not a narrative.	Project Progress ProjectProgress.md in the root — one row per step, PRE-01 to 12.
Packaging & Versioning outputAppPackage/, never deleted; Schema Sync Mode flagged per build.	Repository Hygiene What never reaches the client's remote — and what always does.
AL MCP Server Bootstrap once; prefer its tools over ad hoc terminal use.	BCQuality Snapshot Fetched once, outside the project root; refreshed only on request.
AL Patterns Library Consulted before diagnosing from scratch at Step 07.	AL Standards Guide Fetched at PRE-01. Unreachable means stop and ask — never recall it.

Not every extension needs fourteen steps

LITE

7 steps · 4 documents

Roughly 5–10 AL files. One person driving, one AI model doing the work. No separate sign-off roles.

FULL

14 steps · 19 documents

Past ~10 objects, multiple sign-off roles, the Main / Light / Reasoning split, or heading to AppSource.

Lite reduces process, never the AL rules

Both editions fetch and apply the same OCPF AL Development Standards Guide. The same rules apply to a 5-file extension as to a 50-file one.

FRD + TDD + Sanity Check → **DesignDoc.md**

Testing Feedback Log + Roadmap → **ChangeLog.md**

Multi-model role split → **dropped**

Nothing is lost by switching later. **DesignDoc.md** maps directly onto the full framework's TDD, and **ChangeLog.md** carries straight over. One edition per project — never both at once.

**Simple Enough for
Functional Consultants.**

**Robust Enough for Pro
Developers.**

Created by AJ Ansari, Microsoft MVP